SC4003 Intelligent Agents

Assignment 2: Repeated Prisoners Dilemma Report

By Leila Khaldi (N2504277D)

April 2026

1 Agent Design

1.1 Introduction

The Prisoner's Dilemma is a fundamental problem in game theory where two rational agents, each acting in their own self-interest, fail to achieve the outcome that would be best for both of them. In the standard single-round version, the dominant strategy is always to defect, even though mutual cooperation would yield a higher payoff for both players. However, in a repeated version of the game, cooperation can emerge because players are able to observe and respond to each other's past behaviour, allowing strategies such as punishment and forgiveness to take effect.

This extends the dilemma to three players, which introduces additional complexity. Unlike the two player version, a player cannot selectively punish one opponent without also affecting the other. This means that a single defecting opponent can justify retaliation that inadvertently punishes a cooperative one. The strategy I developed analyses both opponents independently, classifies their behaviour, and makes context-sensitive decisions rather than committing to one fixed rule.

1.2 Agent Rules

When designing my strategy I aimed to not just target one opponent but rather analyse both opponents independently using recent defection rates, classify opponent types and track their cooperation streaks to make contextually sensible decisions. The key design ideas I had were:

1. Be cooperative by default to encourage mutual high payoffs
2. Punish consistent defectors to avoid exploitation
3. Distinguish between opponent types
4. Watch and use opponents recent behaviour

1.3 Strategy Breakdown

```
if (n < 5) return 0;
```

Early rounds: always cooperate because I haven’t yet been able to collect any history from my opponents, as well as create trust. It is also enough rounds to create informed judgements.

```
int opp1Defects = 0, opp2Defects = 0;
for (int i = 0; i < n; i++) {
    if (oppHistory1[i] == 1) opp1Defects++;
    if (oppHistory2[i] == 1) opp2Defects++;
}
double opp1Rate = (double) opp1Defects / n;
double opp2Rate = (double) opp2Defects / n;
```

History defection rates: counts how many times each opponent has defected across the entire match then divides it by the total number of rounds to get a defection rate between 0 and 1. This gave me an overall idea of player behaviour. For example NastyPlayer has a rate of 1.

```
int lookback = Math.min(n, 10);
int opp1Recent = 0, opp2Recent = 0;
for (int i = n - lookback; i < n; i++) {
    if (oppHistory1[i] == 1) opp1Recent++;
    if (oppHistory2[i] == 1) opp2Recent++;
}
double opp1RecentRate = (double) opp1Recent / lookback;
double opp2RecentRate = (double) opp2Recent / lookback;
```

Recent defection rates: does the same as the previous step but only for the last 10 rounds rather than the whole match history. The recent behaviour of the player matters more than their old behaviour.

```
boolean opp1Nasty = (opp1Rate > 0.9);
boolean opp2Nasty = (opp2Rate > 0.9);
boolean opp1Nice = (opp1Rate < 0.05);
boolean opp2Nice = (opp2Rate < 0.05);
```

Opponent types: labels each player nasty, nice or a mix based on their history defection rates.

```
int streak = 0;
for (int i = n-1; i >= 0; i--) {
        if (oppHistory1[i] == 0 && oppHistory2[i] == 0)
            streak++;
        else
      break;
}
```

Cooperation streak: counts backwards through history to find consecutive rounds both opponents cooperated, only unbroken streaks are counted. If both opponents cooperated for 5 or more rounds in a row, cooperate immediately without checking any further.

```
// If both are nasty, defect
if (opp1Nasty && opp2Nasty) return 1;

// If there has been a cooperation streak, safe to cooperate
if (streak >= 5) return 0;

// If both are nice, cooperate
if (opp1Nice && opp2Nice) return 0;

// One nasty and one nice, defect to gain from nice one
if ((opp1Nasty && opp2Nice) || (opp1Nice && opp2Nasty)) return 1;
```

Type-based decisions:

- Both nasty leads to a defect: cooperation against two defectors would otherwise result in 0 points (CDD), defecting gives 2 points (DDD)
- Both nice leads to cooperation: both opponents never defect so mutual cooperation gives everyone 6 points (CCC)
- One nasty one nice leads to a defect: defecting gives 5 points in this situation, while nice would only get 3

```
// Both recently cooperative
if (opp1RecentRate < 0.3 && opp2RecentRate < 0.3) return 0;

// If both have been nasty recently
if (opp1RecentRate > 0.7 && opp2RecentRate > 0.7) return 1;

// If at least one of them have been nasty recently
if (opp1RecentRate > 0.5 || opp2RecentRate > 0.5) return 1;

// Default cooperate
return 0;
```

For the opponents who aren't clearly nasty or nice, I used recent behaviour to decide

- Both recently cooperative (< 30%) = cooperate
- Both recently nasty (> 70%) = defect
- Either recently defecting (> 50%) = defect
- Default = cooperate

2 Evaluation

2.1 Performance against others

Number of Tournaments Played: 1	
Player	**Score**
MyPlayer	172.1347
TolerantPlayer	169.3999
T4TPlayer	163.6313
NicePlayer	157.3701
RandomPlayer	153.4422
NastyPlayer	152.9153
FreakyPlayer	139.1427

Number of Tournaments Played: 5	
Player	**Score**
MyPlayer	170.6543
T4TPlayer	165.5962
TolerantPlayer	164.0533
NicePlayer	154.5837
FreakyPlayer	146.0865
RandomPlayer	147.3155
NastyPlayer	141.3537

Number of Tournaments Played: 10	
Player	**Score**
MyPlayer	167.4559
TolerantPlayer	164.4466
T4TPlayer	162.1073
NicePlayer	153.5008
RandomPlayer	143.1412
NastyPlayer	142.2026
FreakyPlayer	141.9130

Number of Tournaments Played: 50	
Player	**Score**
MyPlayer	168.9300
TolerantPlayer	165.0898
T4TPlayer	164.9334
NicePlayer	154.5323
FreakyPlayer	149.4737
RandomPlayer	144.9854
NastyPlayer	142.2354

Number of Tournaments Played: 100	
Player	**Score**
MyPlayer	167.4724
T4TPlayer	165.4060
TolerantPlayer	164.9041
NicePlayer	153.1476
FreakyPlayer	147.2094
RandomPlayer	145.8301
NastyPlayer	141.5867

Number of Tournaments Played: 500	
Player	**Score**
MyPlayer	167.2703
TolerantPlayer	164.7177
T4TPlayer	164.5117
NicePlayer	153.2917
FreakyPlayer	147.6135
RandomPlayer	144.6974
NastyPlayer	142.1787

Figure 2.1 Player performance rankings based on total scores

In the figure above, it can be seen that my agent which is MyPlayer (*Khaldi_Leila_Player*) outperforms other players in all of the cases. As the number of tournaments played increases there are changes within the rankings for the rest of the players. The key observations are that *T4TPlayer* and *TolerantPlayer* change between second and third position, as well as the bottom three also interchanging positions. My agent stays in first position when the number of tournaments played increases. I have concluded that the more tournaments that are played the higher the chance that my agent outperforms the rest.

As shown in *Figure 2.1*, my agent's score converges and stabilises at approximately 167-168 points after around 50 tournaments. Beyond 50 tournaments the variance becomes negligible, with only a 0.2 point difference observed between 100 and 500 tournament averages. This suggests that 50 tournaments is sufficient to produce a reliable estimate of my agents true expected performance.

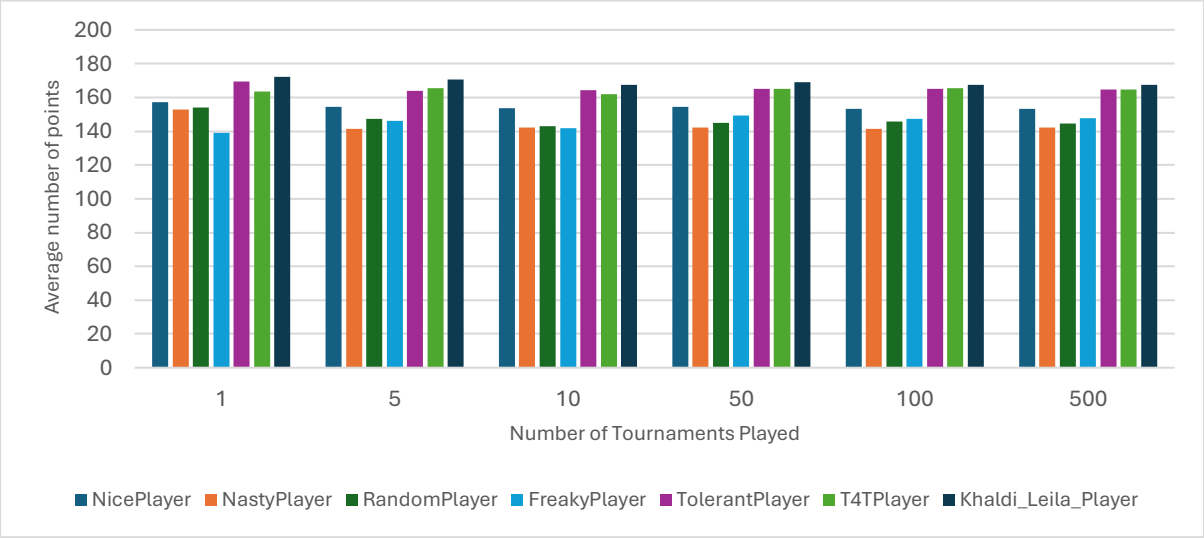

Figure 2.2 Evaluation of performance against other players

Figure 2.2 illustrates the average scores of all seven players across six different tournament sizes ranging from 1 to 500 tournaments. It can be observed that my strategy, consistently achieves the highest average score across all tournament sizes. As the number of tournaments increases the scores of all players begin to stabilise, with the relative rankings between players becoming more consistent. The gap between my player and the second place players *TolerantPlayer* and *T4TPlayer* remains consistent throughout, confirming that the result is not due to random variation but reflects the genuine performance advantage of my strategy. *NastyPlayer* and *RandomPlayer* consistently occupy the lower positions across all tournament sizes, suggesting that purely defective or random strategies are less effective in a repeated cooperative setting.

2.2 Analysis against other players

2.2.1 Against Nice Player

My agent always outperforms *NicePlayer* across all tournament runs. This is because NicePlayer always cooperates without conditions, making it predictable. My agent detects *NicePlayer's* defection rate dropping below 0.05 and correctly classifies it as Nice, cooperating back to achieve the maximum mutual payoff.

2.2.2 Against Nasty Player

My agent consistently scores higher than *NastyPlayer* across all tournament runs. *NastyPlayer* always defects, so my agent detects its defection rate rising above 0.9 and classifies it as Nasty, switching to defect as well.

2.2.3 Against Tolerant Player

My agent performs competitively against *TolerantPlayer*, finishing above it in most tournament runs. *TolerantPlayer* only defects when the majority of combined opponent moves are defections, making it cooperative. My agent recognises this low defection rate and cooperates back, achieving close to the maximum mutual payoff. The competition between them is close because both strategies are cooperative.

2.2.4 Against T4T Player

My agent performs competitively against *T4TPlayer* across all tournament runs. *T4TPlayer* starts by cooperating and then mirrors a randomly chosen opponent's last move. Since my agent cooperates by default in early rounds and maintains cooperation with cooperative opponents, *T4TPlayer* mirrors that cooperation back, resulting in a stable cooperative relationship with high mutual payoffs.

2.2.5 Against Freaky Player

My agent adapts effectively against FreakyPlayer, since *FreakyPlayer* randomly decides at the start of each match to either always cooperate or always defect, my strategy detects this within a few rounds. If *FreakyPlayer* chose to always cooperate, my agent classifies it as Nice and cooperates back. If *FreakyPlayer* chose to always defect, my agent classifies it as Nasty and defects to minimise losses.

2.2.6 Against Random Player

My agent performs reliably above *RandomPlayer* across all tournament runs. Since *RandomPlayer* cooperates and defects with equal probability, its defection rate hovers around 0.5. My agent's recent rate threshold of 0.5 means it cautiously defects against *RandomPlayer* when needed, avoiding the worst payoffs while still benefiting when *RandomPlayer* happens to cooperate.

2.3 Conclusion

I designed a strategy for the three-player repeated Prisoner's Dilemma. The strategy analyses both opponents independently using full and recent defection rates, classifies them as Nasty, Nice or Mixed, and tracks cooperation streaks to make informed decisions each round rather

than following a single fixed rule. As shown in the evaluation my player consistently places first across all tournament runs, with the score converging. The individual player analysis also confirms the strategy handles all six example player effectively.

Overall the strategy demonstrates that a cooperative but adaptive approach outperforms both purely cooperative and purely defective strategies in a repeated three player setting.